Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

15 solution use cases · 67 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Automation Platform
    - Solution use cases (15)
        - Assess the current automation estate
        - Install Automation Platform on RHEL
        - Install Automation Platform on OpenShift
        - Manage Automation Platform as code
        - Onboard teams with RBAC and single sign-on
        - Build and manage execution environments
        - Manage dynamic inventory from sources of truth
        - Scale and tune with automation mesh
        - Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later
        - Move Automation Platform between installation types
        - Back up, restore and design HA and DR
        - Forward logs to a central logging platform
        - Monitor Automation Platform health and capacity
        - Rotate vaulted secrets and platform credentials
        - Collect support data for Red Hat support cases
    - Product use cases
        - Ansible Automation Platform (4)
        - Ansible Automation Platform (automation controller) (7)
        - Ansible Automation Hub (4)
        - Ansible Automation Platform (5)
        - Receptor (1)
        - Red Hat Ansible Automation Platform (configuration as code) (9)
        - Ansible Automation Platform (4)
        - Ansible Automation Platform (3)
        - Ansible Automation Hub (4)
        - Ansible Vault (1)
        - Red Hat Ansible Automation Platform, OpenShift and RHEL platform services (12)
        - Ansible Automation Platform (controller configuration as code) (7)
        - Event-Driven Ansible Controller (3)
        - Ansible execution environments (1)
        - Red Hat Support (2)

Automation Platform

Install, configure as code, upgrade, scale and operate Ansible Automation Platform itself.

Solution use cases

Use Case: Assess the current automation estate

Description	Before moving, consolidating or scaling an Ansible deployment, the team needs facts instead of estimates: how many controllers exist, which version they run, what objects they hold, who uses them and what the jobs depend on. Ansible collects these facts from the platform APIs and produces a baseline that drives sizing, migration scope and clean-up.
Collection	`ansible.controller` (Certified), `ansible.platform` (Certified), `infra.aap_configuration` (Validated), `infra.aap_configuration_extended` (Validated), `infra.ee_utilities` (Validated)
Key content	• `infra.aap_configuration.get_stats` — Get stats • `ansible.controller.export` — export resources from Automation Platform Controller. • `infra.aap_configuration_extended.filetree_create` — An Ansible Role to create a local file tree of objects existing in Controller • `ansible.controller.job_list` — List Automation Platform Controller jobs. • `ansible.controller.controller_api` — Search the API for objects • `infra.ee_utilities.virtualenv_migrate` — Use this role to create a list of python requirements from custom virtualenvs present in your AAP 1.2 cluster, after comparing those with requirements in Default Execution Environment. This role is based on the awx-manage utility and needs an AAP1.2 tower node to gather requirements from and localhost to pull EE and compare those requirements.
Notes	• Use the output as input for Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later and Manage Automation Platform as code. • Exports hold encrypted secret placeholders only. Re-enter or re-source credential secrets in the target; do not assume the export carries them. • Review the usage figures with the owning teams before retiring anything. Low job counts can indicate quarterly or on-demand processes.

Use Case: Install Automation Platform on RHEL

Description	Install Ansible Automation Platform on RHEL with the containerized installer, so the platform gateway, automation controller, automation hub and Event-Driven Ansible controller run as containers on the target hosts. Ansible downloads the installer, generates the installation inventory from variables kept in Git and runs the installation, which makes every environment repeatable.
Collection	`infra.aap_utilities` (Validated), `infra.automation_development_office` (Validated)

Key content	• `infra.aap_utilities.aap_setup_download` — download the latest setup tarball for a version of AAP • `infra.aap_utilities.aap_setup_prepare` — Prepare installation of AAP, OS requirements, unpack tarball, write inventory • `infra.aap_utilities.aap_setup_install` — Trigger installation of AAP given a prepared setup and inventory • `infra.aap_utilities.aap_certs` — Role to install Ansible AAP Certificates • `infra.aap_utilities.aap_manage_containerized_services` — Manage (start/stop/restart/status) the various services for Containerized AAP • `infra.automation_development_office.install_aap` — Install AAP on OpenShift or RHEL via validated infra.aap_utilities (aap_ocp_install / aap_setup_*).
Notes	• Follow with Manage Automation Platform as code, Onboard teams with RBAC and single sign-on and Back up, restore and design HA and DR. • Execution and hop nodes for remote sites belong in the installation inventory; see Scale and tune with automation mesh. • The download and prepare roles also handle the RPM-based installer; check which installer the target release supports before choosing it. • Keep the installer, inventory and `aap_manage_containerized_services` playbooks in one Git repository so that start, stop and restart for patch windows are also automated.

Use Case: Install Automation Platform on OpenShift

Description	Install Ansible Automation Platform on an OpenShift cluster using the Ansible Automation Platform Operator. Ansible subscribes the operator, creates the platform resource with the gateway, controller, hub and Event-Driven Ansible, and keeps the cluster settings in Git, so a new environment is a pull request and a job run.
Collection	`infra.aap_utilities` (Validated), `infra.automation_development_office` (Validated), `redhat.openshift` (Certified)
Key content	• `infra.aap_utilities.aap_ocp_install` — Installs Ansible Automation Platform (AAP) on OpenShift Container Platform (OCP) • `infra.automation_development_office.install_aap` — Install AAP on OpenShift or RHEL via validated infra.aap_utilities (aap_ocp_install / aap_setup_*). • `infra.automation_development_office.ocp_aap_hub_harden` — Harden AAP Automation Hub against shared-Postgres LWLock storms
Notes	• Follow with Manage Automation Platform as code and Back up, restore and design HA and DR. • To move an existing RHEL-based deployment onto OpenShift, use Move Automation Platform between installation types. • Operator channels below 2.5 create separate controller, hub and Event-Driven Ansible resources; from 2.5 the single platform resource is used. Name the channel with a `major.minor` version so the role can tell them apart. • Container groups on the same cluster can run jobs as pods; see Scale and tune with automation mesh.

Use Case: Manage Automation Platform as code

Description	Describe organizations, teams, credentials, projects, inventories, job templates, hub content and Event-Driven Ansible rulebook activations as YAML in Git, and let Ansible create and update them. Changes then follow pull requests, review and promotion from test to production, and the live platform can be compared with Git to find manual changes.
Collection	`infra.aap_configuration` (Validated), `infra.aap_configuration_extended` (Validated), `ansible.platform` (Certified), `ansible.controller` (Certified), `ansible.hub` (Certified), `ansible.eda` (Certified)
Key content	• `infra.aap_configuration.dispatch` — An Ansible Role to call other roles to create objects on Ansible Automation Platform. • `infra.aap_configuration_extended.filetree_create` — An Ansible Role to create a local file tree of objects existing in Controller • `infra.aap_configuration_extended.aap_config_vars` — Load AAP configuration-as-code vars (optionally only changed objects) • `infra.aap_configuration_extended.aap_rules_validation` — An ansible role which audit the declared AAP configuration and validate it against a set of user-defined rules. • `infra.aap_configuration_extended.object_diff` — An Ansible Role to read from a local file tree and create objects in Controller • `infra.aap_configuration.eda_rulebook_activations` — An Ansible Role to create rulebook activations in EDA controller. • `ansible.platform.organization` — Configure a gateway organization • `ansible.controller.job_template` — create, update, or destroy job templates.
Notes	• Pair with Onboard teams with RBAC and single sign-on for the access model and Build and manage execution environments for the images the code refers to. • `object_diff` can remove objects that are not in Git. Run it in report mode first and use `protect_not_empty_orgs` while the model is being adopted. • Use Event-Driven Ansible to trigger the configuration job when a Git push event arrives, instead of polling. • For releases earlier than 2.5, `infra.controller_configuration` holds the previous format; see Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later.

Use Case: Onboard teams with RBAC and single sign-on

Description	Let users sign in with the corporate identity provider and receive access automatically, based on group membership. Each business team gets an organization, a team, and role assignments on the resources it owns, so new teams are onboarded by adding a few lines of configuration rather than by clicking through the UI.
Collection	`ansible.platform` (Certified), `infra.aap_configuration` (Validated), `infra.aap_configuration_extended` (Validated), `redhat.rhbk` (Certified)

Key content	• `ansible.platform.authenticator` — Configure a gateway authenticator. • `ansible.platform.authenticator_map` — Configure a gateway authenticator maps. • `ansible.platform.organization` — Configure a gateway organization • `ansible.platform.team` — Configure a gateway team • `ansible.platform.role_definition` — Configure a gateway role definition. • `ansible.platform.role_team_assignment` — Assign a team permission to a resource or organization. • `infra.aap_configuration.gateway_authenticators` — An Ansible Role to create authenticators on automation platform gateway.
Notes	• Configuration written for Automation Platform 2.4 or earlier keeps LDAP and SAML settings in the controller. `infra.aap_configuration_extended.upgrade_config` converts them into gateway authenticators and authenticator maps; see Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later. • Role assignment roles in `infra.aap_configuration` differ between 2.5 and 2.6 (controller and gateway variants); use the variant that matches the release. • Credentials for the managed systems should come from a secret manager through credential input sources rather than being stored per team. • Put the identity provider itself under automation if it is Red Hat build of Keycloak; the `redhat.rhbk` collection manages realms, clients and users.

Use Case: Build and manage execution environments

Description	Package the Ansible runtime, collections and Python or system dependencies a team needs into a container image, test it, publish it to private automation hub and make it selectable in the controller. Ansible runs the build the same way every time, so jobs behave the same on a laptop, in a pipeline and on an execution node.
Collection	`infra.ee_utilities` (Validated), `ansible.hub` (Certified), `ansible.controller` (Certified), `infra.automation_development_office` (Validated)
Key content	• `infra.ee_utilities.ee_builder` — Excecution enviroment builder • `infra.ee_utilities.create_execution_environment` — Playbook to create custom Execution Environment • `infra.ee_utilities.virtualenv_migrate` — Use this role to create a list of python requirements from custom virtualenvs present in your AAP 1.2 cluster, after comparing those with requirements in Default Execution Environment. This role is based on the awx-manage utility and needs an AAP1.2 tower node to gather requirements from and localhost to pull EE and compare those requirements. • `ansible.hub.ah_ee_repository` — Manage private automation hub execution environment repositories/containers • `ansible.hub.ah_ee_registry` — Manage private automation hub execution environment remote registries • `ansible.controller.execution_environment` — create, update, or destroy Execution Environments in Automation Platform Controller. • `infra.automation_development_office.aap_build_ee` — Build a custom Ansible execution environment image.

Notes	• Pair with Publish collections to automation hub so that collections land in hub before the image build consumes them. • Keep the number of images small: a few purpose-based images (for example one per technology domain) are easier to patch than one per team. • Use Manage Automation Platform as code to register the images, hub repositories and job template assignments. • Disconnected sites push the built images to an internal registry and sync hub content offline.

Use Case: Manage dynamic inventory from sources of truth

Description	Stop maintaining host lists by hand. Automation Platform reads hosts and their attributes from the systems that already know them (a CMDB, a lifecycle manager, a hypervisor manager, an IPAM or a cloud API), groups them by role, environment or location, and refreshes them on a schedule. Job templates then always run against the current estate.
Collection	`ansible.controller` (Certified), `infra.aap_configuration` (Validated), `servicenow.itsm` (Certified), `redhat.insights` (Certified), `redhat.satellite` (Certified), `vmware.vmware` (Certified), `netbox.netbox` (Certified), `amazon.aws` (Certified), `azure.azcollection` (Certified)
Key content	• `ansible.controller.inventory_source` — create, update, or destroy Automation Platform Controller inventory source. • `ansible.controller.inventory_source_update` — Update inventory source(s). • `servicenow.itsm.now` — Inventory source for ServiceNow table records. • `redhat.satellite.foreman` — Foreman inventory source • `vmware.vmware.vms` — Create an inventory containing VMware VMs • `netbox.netbox.nb_inventory` — NetBox inventory source • `amazon.aws.aws_ec2` — EC2 inventory source • `azure.azcollection.azure_rm` — Azure Resource Manager inventory plugin
Notes	• Define inventories, sources and schedules in Git with Manage Automation Platform as code. • Constructed inventories can combine several sources, for example ServiceNow ownership data with Satellite content data. • Use Event-Driven Ansible with `servicenow.itsm.records` or cloud event sources to refresh inventory when records change instead of waiting for the schedule. • Large inventories benefit from caching in the plugin, filtered queries at the source and splitting by scope; see Scale and tune with automation mesh. • Red Hat Insights (`redhat.insights.insights`) can serve as an additional source for RHEL systems that are registered with it but not in Satellite.

Use Case: Scale and tune with automation mesh

Description	Run automation near the systems it manages and grow capacity without growing the control plane. Automation mesh connects control nodes to execution nodes through Receptor, optionally across firewalls and DMZs using hop nodes. Instance groups route each job template to the right nodes, and job settings keep large runs fast.
Collection	`ansible.receptor` (Certified), `ansible.controller` (Certified), `infra.aap_configuration` (Validated)

Key content	• `ansible.receptor.setup` — Installs and configures receptor nodes. • `ansible.receptor.podman` — Installs and configures podman for use for receptor nodes. • `ansible.controller.instance` — create, update, or destroy Automation Platform Controller instances. • `ansible.controller.instance_group` — create, update, or destroy Automation Platform Controller instance groups. • `ansible.controller.job_template` — create, update, or destroy job templates. • `infra.aap_configuration.controller_instance_groups` — An Ansible Role to create instance groups on Ansible Controller.
Notes	• Prefer control-only nodes once execution nodes exist, so job load does not slow the API. • `ansible.receptor` configures the Receptor service and peering; the installer remains the reference for nodes it manages, so do not configure the same node both ways. • On OpenShift, container groups run jobs as pods; combine them with the mesh for hybrid estates. See Install Automation Platform on OpenShift. • Before adding nodes, check the playbooks themselves: serial execution, long fact gathering and unneeded loops cost more than missing capacity.

Use Case: Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later

Description	Move an existing Ansible Tower or Automation Platform 2.4 deployment to Automation Platform 2.5 or later, where the platform gateway fronts the controller, hub and Event-Driven Ansible. Ansible exports the objects, converts the authentication settings to gateway authenticators, rebuilds custom environments as execution environments and applies everything to a clean target, so the old and new platforms can run side by side until cut-over.
Collection	`ansible.controller` (Certified), `ansible.platform` (Certified), `infra.aap_configuration` (Validated), `infra.aap_configuration_extended` (Validated), `infra.ee_utilities` (Validated), `infra.aap_utilities` (Validated)
Key content	• `ansible.controller.export` — export resources from Automation Platform Controller. • `ansible.controller.import` — import resources into Automation Platform Controller. • `infra.aap_configuration_extended.filetree_create` — An Ansible Role to create a local file tree of objects existing in Controller • `infra.aap_configuration_extended.upgrade_config` — An Ansible Role to migrate configuration files from AAP <= 2.4 format to AAP >= 2.5 • `infra.aap_configuration.dispatch` — An Ansible Role to call other roles to create objects on Ansible Automation Platform. • `infra.ee_utilities.virtualenv_migrate` — Use this role to create a list of python requirements from custom virtualenvs present in your AAP 1.2 cluster, after comparing those with requirements in Default Execution Environment. This role is based on the awx-manage utility and needs an AAP1.2 tower node to gather requirements from and localhost to pull EE and compare those requirements. • `ansible.platform.authenticator` — Configure a gateway authenticator. • `infra.aap_utilities.aap_backup` — Role to backup the Ansible Automation Platform

Notes	• API paths change with the gateway: controller endpoints are served under the gateway URL (for example `/api/controller/v2/`). Update scripts and external tools. • Secrets in credentials are not exported in clear text. Plan to re-supply them from the vault or external secret manager, and consider moving them to credential input sources. • If the source is already on a containerized or operator-based 2.6 deployment and only the platform changes, use Move Automation Platform between installation types. • Managing the target as code from day one avoids repeating the migration for the next upgrade: Manage Automation Platform as code. • Event-Driven Ansible rulebook activations and decision environments are recreated on the target; review rulebook sources and credentials after the move.

Use Case: Move Automation Platform between installation types

Description	Re-platform a running Automation Platform deployment without rebuilding it by hand, for example from the RPM installer to the containerized installer or to OpenShift. Ansible exports the data of the controller, hub (with its content), gateway and Event-Driven Ansible into one artifact, checks it, and imports it into a prepared target of the same version.
Collection	`ansible.aap_snapshot` (Certified)
Key content	• `ansible.aap_snapshot.artifact_export` — Preflight Tasks • `ansible.aap_snapshot.artifact_import` — Preflight • `ansible.aap_snapshot.artifact_verify` — Verify artifact • `ansible.aap_snapshot.aap_component_info` — Discover AAP component information from RPM installations • `ansible.aap_snapshot.validate_migration_artifact` — Validate an AAP migration artifact • `ansible.aap_snapshot.preflight` — Pre-migration validation and compatibility checks
Notes	• Cross-version imports are not supported. To change version, export from a source running the target version, or upgrade first. • The target keeps its own configuration. Anything environment-specific (identity provider, certificates, mesh nodes) must be done in advance. • Use the same workflow as a portable backup between installations of the same type; see Back up, restore and design HA and DR. • For moves from Tower or 2.4, which change the architecture, use Migrate from Ansible Tower or AAP 2.4 to AAP 2.5 and later instead.

Use Case: Back up, restore and design HA and DR

Description	Make the automation platform itself recoverable. Ansible takes regular backups of the platform database and settings, stores them off-host, and runs restore tests on a standby. The same exercise records which components have to be redundant (gateway, controller, hub, database, execution nodes) and what the recovery steps are.
Collection	`infra.aap_utilities` (Validated), `ansible.aap_snapshot` (Certified), `ansible.controller` (Certified)

Key content	• `infra.aap_utilities.aap_backup` — Role to backup the Ansible Automation Platform • `infra.aap_utilities.aap_restore` — Role to restore the Ansible Automation Platform • `infra.aap_utilities.aap_manage_containerized_services` — Manage (start/stop/restart/status) the various services for Containerized AAP • `ansible.aap_snapshot.artifact_export` — Preflight Tasks • `ansible.controller.schedule` — create, update, or destroy Automation Platform Controller schedules. • `ansible.controller.job_template` — create, update, or destroy job templates.
Notes	• Restore into the same Automation Platform version, and keep the installation inventory and its secrets safe, because a restore uses them. • On OpenShift, backups are taken with the operator's backup custom resource and cluster tools, not with the installer-based roles. See Install Automation Platform on OpenShift. • Execution nodes hold no platform data, so they can be rebuilt from automation. See Scale and tune with automation mesh. • Back up before any rotation or upgrade: Rotate secrets and credentials. • Pair with Monitor Automation Platform health and capacity and Event-Driven Ansible to restart failed services automatically.

Use Case: Forward logs to a central logging platform

Description	Give the security and operations teams the platform's logs in the tools they already use. Automation Platform can stream structured job events, activity stream entries and system logs to an external aggregator such as Splunk, Logstash or a generic HTTP or syslog target. Ansible applies and verifies the logging settings so every instance sends the same data to the same place.
Collection	`ansible.controller` (Certified), `ansible.platform` (Certified), `infra.aap_configuration` (Validated)
Key content	• `ansible.controller.settings` — Modify Automation Platform Controller settings. • `ansible.platform.settings` — Modify automation platform gateway settings. • `infra.aap_configuration.controller_settings` — An Ansible Role to create settings on Ansible Controller. • `infra.aap_configuration.gateway_settings` — An Ansible Role to create settings on automation platform gateway. • `ansible.controller.job_template` — create, update, or destroy job templates.
Notes	• Large volumes of job events can be significant. Filter at the source with the log level and enabled log types, and size the receiving side before turning on verbose output. • Pair with Monitor Automation Platform health and capacity: logs show what happened, metrics show how loaded the platform is. • Use Event-Driven Ansible to react to logged failures, for example to open an incident or retry a job, instead of only alerting. • Manage the settings alongside the other configuration: Manage Automation Platform as code.

Use Case: Monitor Automation Platform health and capacity

Description	Know whether the automation platform is healthy and has room to run the next job. Ansible collects metrics and service status from every component, feeds them to the monitoring tool the operations team already uses, installs dashboards and alerts, and can restart failed services on its own.
Collection	`infra.aap_configuration` (Validated), `infra.aap_utilities` (Validated), `ansible.controller` (Certified), `infra.automation_development_office` (Validated), `ansible.eda` (Certified)
Key content	• `infra.aap_configuration.get_stats` — Get stats • `infra.aap_utilities.aap_manage_containerized_services` — Manage (start/stop/restart/status) the various services for Containerized AAP • `ansible.controller.instance` — create, update, or destroy Automation Platform Controller instances. • `ansible.controller.instance_group` — create, update, or destroy Automation Platform Controller instance groups. • `infra.automation_development_office.grafana_upload_dashboards` — Role to upload Grafana dashboards. • `infra.automation_development_office.grafana_upload_alerts` — Upload Grafana alert JSON/JSON.j2 files from folder sources • `ansible.eda.alertmanager` — Receive events via a webhook from alertmanager or a compatible alerting system.
Notes	• Combine with Forward logs to a central logging platform for the full picture. • Capacity numbers come from instance and instance group settings; see Scale and tune with automation mesh for acting on them. • Keep remediation jobs conservative: restart services and re-sync, but leave database or storage failures to people. • The `get_stats` playbook expects to run on or against the platform with an administrator account attached to the job template.

Use Case: Rotate vaulted secrets and platform credentials

Description	Change the secrets that automation depends on on a schedule or after an incident, without hand-editing hundreds of files. Ansible re-encrypts every vaulted variable in the inventory repositories under a new vault password and updates the matching vault credential in the controller. The same approach rotates the platform's own database passwords and encryption keys.
Collection	`infra.ansible_rekey_variables` (Validated), `ansible.scm` (Certified), `ansible.controller` (Certified), `infra.aap_utilities` (Validated)

Key content	• `infra.ansible_rekey_variables.rekey_variable` — Inventory vault password change for variables • `ansible.scm.git_retrieve` — Retrieve a repository from a distant location and make it available on the execution node • `ansible.scm.git_publish` — Publish changes from a repository available on the execution node to a distant location • `ansible.controller.credential` — create, update, or destroy Automation Platform Controller credential. • `infra.aap_utilities.aap_password_rotate` — Rotate passwords across AAP 2.7 components • `infra.aap_utilities.aap_secret_rotate` — Rotate SECRET_KEY across AAP 2.7 components
Notes	• Rekeying rewrites many files. Do it from a clean branch, review only that the diff consists of vault blocks, and merge quickly to avoid conflicts. • Password and key rotation are separate operations: passwords change authentication, keys change how stored fields are encrypted. Do them in separate windows. • The platform rotation roles target recent releases and the containerized and operator deployments; check the role documentation for the supported versions before use. • Prefer secret managers through credential input sources for the long term, so that fewer secrets need to be vaulted in files at all. See Manage Automation Platform as code.

Use Case: Collect support data for Red Hat support cases

Description	Cut the time spent on the first round of questions in a support case. One job collects what Red Hat Support usually asks for (component API output, sos reports from RHEL hosts, must-gather from OpenShift), packages it, can create the case, and uploads the files to it.
Collection	`infra.support_assist` (Validated)
Key content	• `infra.support_assist.aap_api_gather` — Auto-generated specs for aap_api_gather role - main entry point • `infra.support_assist.aap_api_token` — Auto-generated specs for aap_api_token role - main entry point • `infra.support_assist.sos_report_rhel` — RHEL Host SOS Report Pipeline • `infra.support_assist.ocp_must_gather` — Auto-generated specs for ocp_must_gather role - main entry point • `infra.support_assist.rh_case_create` — Red Hat Support Case Creation - API Only Pipeline • `infra.support_assist.rh_case_update` — Add comment(s) or attach file(s) to Red Hat Support Case
Notes	• Review what the archives contain before uploading. sos reports and must-gather output can include host names, addresses and configuration; apply the customer's data-sharing rules. • Run Monitor Automation Platform health and capacity first: its output often shows where to collect from. • For sos reports there is also a generic `infra.support_assist.sos_report` playbook, which carries a deprecation notice; prefer the RHEL and OpenShift variants. • Use Event-Driven Ansible to start the diagnostics workflow when a critical alert fires, so the data is captured at the moment of failure.

Product use cases

Grouped from the modules of each certified or validated collection. Key content lists the components each use case supports, with the modules that manage them (create, update, delete) and query them (_info).

Ansible Automation Platform `ansible.aap_snapshot`

Certified Red Hat · version 1.1.2 · updated 2026-10-14 · 2 modules → 4 use cases

Use Case: Check readiness before migration

Description	Run preflight checks, discover installed AAP components and normalize inventory groups across deployment types.
Collection	`ansible.aap_snapshot` (Certified, Red Hat)
Key content	**Pre-migration checks** role `preflight` **Inventory group normalization** role `common` **AAP component discovery** `aap_component_info` **AAP version parsing** filter `aap_version`

Use Case: Export and import component data

Description	Export and import Automation Controller, EDA, Gateway and Hub data and the PostgreSQL databases behind them.
Collection	`ansible.aap_snapshot` (Certified, Red Hat)
Key content	**Automation Controller data** role `automationcontroller` **Event-Driven Ansible data** role `automationeda` **Automation Gateway data** role `automationgateway` **Automation Hub and Pulp content** role `automationhub` **PostgreSQL databases** role `postgresql` **Export artifact** playbook `artifact_export` **Import artifact** playbook `artifact_import`

Use Case: Manage and verify migration artifacts

Description	Manage the artifact lifecycle and validate migration artifacts.
Collection	`ansible.aap_snapshot` (Certified, Red Hat)
Key content	**Artifact lifecycle** role `artifact` **Artifact validation** `validate_migration_artifact` **Verify artifact** playbook `artifact_verify` **Validate artifact** playbook `common.validate_artifact`

Use Case: Operate platform services during migration

Description	Start and stop services, manage component versions, Receptor mesh, Performance Co-Pilot and OpenShift operator-based deployments.
Collection	`ansible.aap_snapshot` (Certified, Red Hat)
Key content	**Component operations** role `operations` **Receptor mesh** role `receptor` **Performance Co-Pilot** role `pcp` **OpenShift cluster operations** role `ocp_utils` **Start EDA services** playbook `common.start_services` **Stop monitoring services** playbook `common.stop_services`

Ansible Automation Platform (automation controller) `ansible.controller`

Certified Red Hat · version 4.8.11 · updated 2026-09-30 · 44 modules → 7 use cases

Use Case: Launch and monitor jobs and workflows

Description	Launch job templates, bulk jobs and workflows, wait for completion, cancel or approve runs, and list job results.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Job launch** `job_launch` **Bulk job launch** `bulk_job_launch` **Wait for job** `job_wait` **Cancel job** `job_cancel` **Job listing** `job_list` **Workflow launch** `workflow_launch` **Wait for workflow node** `workflow_node_wait` **Workflow approvals** `workflow_approval` **Ad hoc commands** `ad_hoc_command` **Cancel ad hoc command** `ad_hoc_command_cancel` **Wait for ad hoc command** `ad_hoc_command_wait`

Use Case: Define job templates, workflows and schedules

Description	Create job templates, workflow templates and their nodes, schedules, labels and notification templates, and generate schedule recurrence rules.
Collection	`ansible.controller` (Certified, Red Hat)

Key content	**Job templates** `job_template` **Workflow job templates** `workflow_job_template` **Workflow nodes** `workflow_job_template_node` **Schedules** `schedule` **Labels** `label` **Notification templates** `notification_template` **Schedule rrule lookup** lookup `schedule_rrule` **Schedule rruleset lookup** lookup `schedule_rruleset`

Use Case: Manage projects and execution environments

Description	Create projects from source control, trigger project updates and register execution environments.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Projects** `project` **Project updates** `project_update` **Execution environments** `execution_environment`

Use Case: Manage inventories, hosts and dynamic sources

Description	Create inventories, groups and hosts (including in bulk), configure inventory sources and trigger source syncs, and build inventory from the controller API.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Inventories** `inventory` **Groups** `group` **Hosts** `host` **Bulk host create** `bulk_host_create` **Bulk host delete** `bulk_host_delete` **Inventory sources** `inventory_source` **Inventory source updates** `inventory_source_update` **Controller dynamic inventory** inventory `controller`

Use Case: Manage credentials and secrets

Description	Create credentials, custom credential types and credential input sources for external secret managers.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Credentials** `credential` **Credential types** `credential_type` **Credential input sources** `credential_input_source`

Use Case: Manage organizations, users and access control

Description	Create organizations, teams and users, define role definitions and assign them to users and teams.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Organizations** `organization` **Teams** `team` **Users** `user` **Role definitions** `role_definition` **User role assignments** `role_user_assignment` **Team role assignments** `role_team_assignment`

Use Case: Administer the controller platform

Description	Manage instances and instance groups, system settings and licensing, export or import configuration, and query the API with lookups.
Collection	`ansible.controller` (Certified, Red Hat)
Key content	**Instances** `instance` **Instance groups** `instance_group` **Settings** `settings` **License** `license` **Subscriptions** `subscriptions` **Export configuration** `export` **Import configuration** `import` **Collection metadata** `controller_meta` **Controller API lookup** lookup `controller_api`

Ansible Automation Hub `ansible.hub`

Certified Red Hat · version 1.1.2 · updated 2026-10-01 · 21 modules → 4 use cases

Use Case: Build, upload and approve collections

Description	Build collection tarballs, upload artifacts, approve and update or remove collections.
Collection	`ansible.hub` (Certified, Red Hat)
Key content	**Collection builds** `ah_build` **Collection uploads** `ah_collection_upload` **Collection approvals** `ah_approval` **Collections** `ah_collection` **Namespaces** `ah_namespace`

Use Case: Configure collection repositories

Description	Create repositories and remotes and sync collection repositories.
Collection	`ansible.hub` (Certified, Red Hat)
Key content	**Collection repositories** `collection_repository` **Collection remotes** `collection_remote` **Collection repository syncs** `collection_repository_sync`

Use Case: Manage execution environment images

Description	Manage execution environment registries, repositories and images and run registry indexing and sync.
Collection	`ansible.hub` (Certified, Red Hat)
Key content	**EE registries** `ah_ee_registry` **EE registry indexing** `ah_ee_registry_index` **EE registry syncs** `ah_ee_registry_sync` **EE repositories** `ah_ee_repository` **EE repository syncs** `ah_ee_repository_sync` **EE images** `ah_ee_image`

Use Case: Manage users, teams and access

Description	Manage users, groups, teams, roles, role assignments and tokens.
Collection	`ansible.hub` (Certified, Red Hat)
Key content	**Users** `ah_user` **Groups** `ah_group` **Teams** `ah_team` **Roles** `ah_role` **Group roles** `group_roles` **Team roles** `team_roles` **Tokens** `ah_token`

Ansible Automation Platform `ansible.platform`

Certified Red Hat · version 2.7.20260930 · updated 2026-09-30 · 31 modules → 5 use cases

Use Case: Manage organizations, users, teams and access

Description	Create gateway organizations, users, teams, role definitions and role assignments, and manage tokens and applications.
Collection	`ansible.platform` (Certified, Red Hat)

Key content	**Organizations** `organization` **Teams** `team` **Users** `user` **Role definitions** `role_definition` **User role assignments** `role_user_assignment` **Team role assignments** `role_team_assignment` **Tokens** `token` **Applications** `application`

Use Case: Configure authentication

Description	Configure authenticators, authenticator maps and user authenticators for platform sign-in.
Collection	`ansible.platform` (Certified, Red Hat)
Key content	**Authenticators** `authenticator` **Authenticator maps** `authenticator_map` **User authenticators** `authenticator_user`

Use Case: Configure the gateway and its services

Description	Configure gateway settings, services, nodes, clusters, routes, ports, certificates and feature flags.
Collection	`ansible.platform` (Certified, Red Hat)
Key content	**Gateway settings** `settings` **Feature flags** `feature_flag` **Services** `service` **Service clusters** `service_cluster` **Service nodes** `service_node` **Service types** `service_type` **Service keys** `service_key` **Custom routes** `route` **UI plugin routes** `ui_plugin_route` **HTTP ports** `http_port` **CA certificates** `ca_certificate`

Use Case: Manage automation controller content

Description	Manage controller inventories, hosts, groups, job templates, labels, credential types and instance groups, and list jobs.
Collection	`ansible.platform` (Certified, Red Hat)

Key content	**Inventory sources** `inventory_source` **Hosts** `host` **Inventory groups** `group` **Job templates** `job_template` **Jobs** `job_list` **Labels** `label` **Credential types** `controller_credential_type` **Credential input sources** `credential_input_source` **Instance groups** `instance_group`

Use Case: Call the platform API

Description	Query the platform API from playbooks and connect over HTTP.
Collection	`ansible.platform` (Certified, Red Hat)
Key content	**HTTP connection** connection `http` **Gateway API lookup** lookup `gateway_api`

Receptor `ansible.receptor`

Certified Red Hat · version 2.0.9 · updated 2026-05-15 · 0 modules → 1 use cases

Use Case: Deploy Receptor nodes

Description	Install and configure Receptor nodes and Podman for them.
Collection	`ansible.receptor` (Certified, Red Hat)
Key content	**Receptor nodes** role `setup` **Podman** role `podman`

Red Hat Ansible Automation Platform (configuration as code)

`infra.aap_configuration`

Validated Red Hat · version 4.10.0 · updated 2026-09-15 · 0 modules → 9 use cases

Use Case: Configure the whole platform from code

Description	Apply a complete configuration to Automation Platform with the dispatch role and playbooks, and share global variables and ansible.cfg settings.
Collection	`infra.aap_configuration` (Validated, Red Hat)

Key content	**Dispatch role** role `dispatch` **Global variables** role `global_vars` **ansible.cfg generation** role `ansible_config` **Collection dependency check** role `meta_dependency_check` **Post-installation configuration** playbook `configure_aap` **Organization assignment** playbook `set_organization` **Object renaming** playbook `rename_objects` **Platform statistics** playbook `get_stats`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Manage organizations, users, teams and RBAC in the controller

Description	Create organizations, users and teams and grant access with roles and role definitions in automation controller.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Controller organizations** role `controller_organizations` **Controller users** role `controller_users` **Controller teams** role `controller_teams` **Controller roles** role `controller_roles` **Controller role definitions** role `controller_role_definitions` **Controller team role assignments** role `controller_role_team_assignments` **Controller user role assignments** role `controller_role_user_assignments` **Controller settings** role `controller_settings` **Controller licence** role `controller_license` **Labels** role `controller_labels` **OAuth applications** role `controller_applications`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Manage credentials and execution environments in the controller

Description	Define credential types, credentials and input sources, execution environments, instances and instance groups.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Credential types** role `controller_credential_types` **Credentials** role `controller_credentials` **Credential input sources** role `controller_credential_input_sources` **Execution environments** role `controller_execution_environments` **Instances** role `controller_instances` **Instance groups** role `controller_instance_groups`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Manage projects, inventories and hosts in the controller

Description	Create and sync projects, inventories, inventory sources, hosts and host groups, including bulk host creation.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Projects** role `controller_projects` **Project updates** role `controller_project_update` **Inventories** role `controller_inventories` **Inventory sources** role `controller_inventory_sources` **Inventory source updates** role `controller_inventory_source_update` **Hosts** role `controller_hosts` **Host groups** role `controller_host_groups` **Bulk host creation** role `controller_bulk_host_create`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Define and run jobs and workflows in the controller

Description	Create job and workflow templates, schedules and notifications, then launch, monitor and cancel jobs and ad hoc commands.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Job templates** role `controller_job_templates` **Workflow job templates** role `controller_workflow_job_templates` **Schedules** role `controller_schedules` **Notification templates** role `controller_notification_templates` **Job launches** role `controller_job_launch` **Workflow launches** role `controller_workflow_launch` **Bulk job launches** role `controller_bulk_job_launch` **Job cancellation** role `controller_jobs_cancel` **Ad hoc commands** role `controller_ad_hoc_command` **Ad hoc command cancellation** role `controller_ad_hoc_command_cancel` **Asynchronous task status** role `collect_async_status`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Configure Event-Driven Ansible

Description	Create EDA users, tokens, credentials, decision environments, projects, event streams and rulebook activations.
Collection	`infra.aap_configuration` (Validated, Red Hat)

Key content	**EDA users** role `eda_users` **Controller tokens** role `eda_controller_tokens` **Credential types** role `eda_credential_types` **Credentials** role `eda_credentials` **Credential input sources** role `eda_credential_input_sources` **Decision environments** role `eda_decision_environments` **Projects** role `eda_projects` **Event streams** role `eda_event_streams` **Rulebook activations** role `eda_rulebook_activations`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Configure the platform gateway

Description	Manage gateway organizations, users, teams, authenticators and maps, role assignments, settings, applications, services, routes, ports and service clusters.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Gateway organizations** role `gateway_organizations` **Gateway users** role `gateway_users` **Gateway teams** role `gateway_teams` **Authenticators** role `gateway_authenticators` **Authenticator maps** role `gateway_authenticator_maps` **Role definitions** role `gateway_role_definitions` **Team role assignments** role `gateway_role_team_assignments` **User role assignments** role `gateway_role_user_assignments` **Gateway settings** role `gateway_settings` **OAuth applications** role `gateway_applications` **Services** role `gateway_services` **Service clusters** role `gateway_service_clusters` **Service nodes** role `gateway_service_nodes` **Service keys** role `gateway_service_keys` **Routes** role `gateway_routes` **HTTP ports** role `gateway_http_ports`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Manage Automation Hub users and groups

Description	Create Hub users, groups, group roles and role permissions.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Users** role `hub_user` **Groups** role `hub_group` **Group roles** role `hub_group_roles` **Role permissions** role `hub_role`

Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Use Case: Curate collections and execution environment images in Automation Hub

Description	Manage namespaces, collection remotes, repositories and publishing, and execution environment registries, repositories and images.
Collection	`infra.aap_configuration` (Validated, Red Hat)
Key content	**Namespaces** role `hub_namespace` **Collections** role `hub_collection` **Collection remotes** role `hub_collection_remote` **Collection repositories** role `hub_collection_repository` **Collection repository sync** role `hub_collection_repository_sync` **Collection publishing** role `hub_publish` **EE registries** role `hub_ee_registry` **EE registry indexing** role `hub_ee_registry_index` **EE registry sync** role `hub_ee_registry_sync` **EE repositories** role `hub_ee_repository` **EE repository sync** role `hub_ee_repository_sync` **EE images** role `hub_ee_image`
Notes	• Roles take lists of objects as variables; the dispatch role calls the other roles in the right order to configure a whole platform.

Ansible Automation Platform `infra.aap_configuration_extended`

Validated Red Hat · version 4.12.1 · updated 2026-10-15 · 3 modules → 4 use cases

Use Case: Export controller configuration to code

Description	Create a file tree of existing controller objects, export job templates and workflows with related objects, and flatten the output.
Collection	`infra.aap_configuration_extended` (Validated, Red Hat)
Key content	**File tree export** role `filetree_create` **Export job template and related objects** playbook `export_job_template_related` **Export workflow template and related objects** playbook `export_workflow_job_template_related` **Flatten export output** playbook `flatten_filetree_create_output` **Workflow related objects** lookup `workflow_related_graph` **Export field variable reference** filter `filetree_as_var` **Export field variable name** filter `filetree_var_name` **Survey cleanup** filter `filetree_surveys_cleanup`

Use Case: Apply configuration as code

Description	Read a file tree or variables and create objects in the controller, configure it after installation and sync collections offline.
Collection	`infra.aap_configuration_extended` (Validated, Red Hat)
Key content	**File tree import** role `filetree_read` **Configuration variables** `aap_config_vars` **Import file tree** playbook `import_filetree` **Post-install controller configuration** playbook `configure_controller` **Offline collection sync** role `offline_sync`

Use Case: Detect drift and validate configuration

Description	Compare controller objects with code and audit declared configuration against rules.
Collection	`infra.aap_configuration_extended` (Validated, Red Hat)
Key content	**Object diff** role `object_diff` **Controller diff** `controller_export_diff` **Controller object diff lookup** lookup `controller_object_diff` **Configuration rule validation** role `aap_rules_validation`

Use Case: Refactor and upgrade configuration

Description	Rename objects, set organizations, upgrade configuration from AAP 2.4 to 2.5 format, report stats and format YAML.
Collection	`infra.aap_configuration_extended` (Validated, Red Hat)
Key content	**Rename objects** playbook `rename_objects` **Set organization** playbook `set_organization` **Upgrade configuration** playbook `upgrade_config` **Configuration upgrade** role `upgrade_config` **Statistics** playbook `get_stats` **YAML formatting** `format_yaml`

Ansible Automation Platform `infra.aap_utilities`

Validated Red Hat · version 4.0.0 · updated 2026-09-11 · 0 modules → 3 use cases

Use Case: Install Ansible Automation Platform

Description	Download the setup bundle, prepare the system and inventory, run the installer, install on OpenShift and set up certificates and a Git SSH server.
Collection	`infra.aap_utilities` (Validated, Red Hat)

Key content	**Setup download** role `aap_setup_download` **Installation preparation** role `aap_setup_prepare` **Installation** role `aap_setup_install` **OpenShift installation** role `aap_ocp_install` **Certificates** role `aap_certs` **Git over SSH server** role `git_ssh_setup` **Kerberos for Windows management** role `kerberos`

Use Case: Operate Ansible Automation Platform

Description	Start, stop and check containerized services and rotate passwords and secret keys.
Collection	`infra.aap_utilities` (Validated, Red Hat)
Key content	**Containerized services** role `aap_manage_containerized_services` **Password rotation** role `aap_password_rotate` **Secret key rotation** role `aap_secret_rotate`

Use Case: Back up, restore and remove Ansible Automation Platform

Description	Back up and restore the platform and remove it.
Collection	`infra.aap_utilities` (Validated, Red Hat)
Key content	**Backup** role `aap_backup` **Restore** role `aap_restore` **Removal** role `aap_remove`

Ansible Automation Hub `infra.ah_configuration`

Validated Red Hat · version 2.1.0 · updated 2024-10-10 · 0 modules → 4 use cases

Use Case: Manage collections and repositories

Description	Configure collection remotes and repositories, sync them, publish and update collections and sync collections offline.
Collection	`infra.ah_configuration` (Validated, Red Hat)
Key content	**Collection remotes** role `collection_remote` **Collection repositories** role `collection_repository` **Collection repository syncs** role `collection_repository_sync` **Repositories** role `repository` **Repository syncs** role `repository_sync` **Collections** role `collection` **Namespaces** role `namespace` **Collection publishing** role `publish` **Offline sync** role `offline_sync`

Use Case: Manage execution environment content

Description	Configure execution environment registries, namespaces, repositories and images and run index and sync operations.
Collection	`infra.ah_configuration` (Validated, Red Hat)
Key content	**EE registries** role `ee_registry` **EE registry indexing** role `ee_registry_index` **EE registry syncs** role `ee_registry_sync` **EE namespaces** role `ee_namespace` **EE repositories** role `ee_repository` **EE repository syncs** role `ee_repository_sync` **EE images** role `ee_image`

Use Case: Manage users, groups and permissions

Description	Create users, groups, group roles and role permissions in Automation Hub.
Collection	`infra.ah_configuration` (Validated, Red Hat)
Key content	**Users** role `user` **Groups** role `group` **Group roles** role `group_roles` **Role permissions** role `role`

Use Case: Configure clients and run the full configuration

Description	Generate ansible.cfg files for Automation Hub servers and run every configuration role for which variables are defined.
Collection	`infra.ah_configuration` (Validated, Red Hat)
Key content	**ansible.cfg files** role `ansible_config` **Configuration dispatch** role `dispatch`

Ansible Vault `infra.ansible_rekey_variables`

Validated Red Hat · version 1.1.4 · updated 2024-05-06 · 0 modules → 1 use cases

Use Case: Change the vault password of inventory variables

Description	Re-encrypt inventory variables with a new vault password.
Collection	`infra.ansible_rekey_variables` (Validated, Red Hat)
Key content	**Rekey variables** role `rekey_variable`

Red Hat Ansible Automation Platform, OpenShift and RHEL platform services

`infra.automation_development_office`

Validated Red Hat · version 2.0.0 · updated 2026-10-06 · 1 modules → 12 use cases

Use Case: Install and configure Ansible Automation Platform

Description	Install AAP on OpenShift or RHEL, configure its objects, build execution environment images and generate bootstrap content for new components.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**AAP installation** role `install_aap` **AAP configuration** role `aap_configuration` **Execution environment image build** role `aap_build_ee` **Automation Hub hardening** role `ocp_aap_hub_harden` **Controller objects bootstrap** role `bootstrap_controller` **Bootstrap variable flattening** role `bootstrap_flatten_vars` **Bootstrap framework defaults** role `bootstrap_framework_defaults` **Bootstrap environment variables** role `bootstrap_generate_env_vars` **Bootstrap playbook repository** role `bootstrap_generate_playbook_repo` **Bootstrap component resolution** role `bootstrap_resolve_component`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Manage AWS resources

Description	Copy AMIs between regions, create, modify, query and delete EC2 instances, and set or read S3 bucket policies.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**AMI copies** `ec2_ami_copy` **EC2 instance creation** playbook `aws.ec2.ado_ec2_instance_create` **EC2 instance modification** playbook `aws.ec2.ado_ec2_instance_modify` **EC2 instance information** playbook `aws.ec2.ado_ec2_instance_info` **EC2 instance deletion** playbook `aws.ec2.ado_ec2_instance_delete` **S3 bucket policy (inline)** playbook `aws.s3.ado_s3_bucket_policy_set` **S3 bucket policy (from file)** playbook `aws.s3.ado_s3_bucket_policy_set_from_file` **S3 bucket policy information** playbook `aws.s3.ado_s3_bucket_policy_info`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Operate and harden RHEL hosts

Description	Patch hosts, manage repositories, services, mounts and cron schedules, call RHEL system roles, gather facts and apply DISA STIG compliance.

Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Patching** role `rhel_patching` **Repositories** role `rhel_repos` **Services** role `rhel_services_management` **Filesystem mounts** role `rhel_mount` **Special cron schedules** role `rhel_cron` **RHEL facts** role `rhel_facts` **RHEL system roles wrapper** role `rhel_ext_system_roles` **DISA STIG compliance** role `rhel_stig_cac`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Install and configure Red Hat Satellite

Description	Install Satellite and Capsule servers, configure Satellite, manage content views and register RHEL hosts.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Satellite installation** role `satellite_install` **Capsule installation** role `capsule_install` **Satellite configuration** role `satellite_config` **Content views** role `satellite_content_view` **Satellite OIDC login** role `satellite_oidc` **Host registration to Satellite** role `rhel_sat_reg`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Set up identity management and directory services

Description	Deploy IdM servers, replicas, clients and DNS, establish IdM and Active Directory trust, and install 389 Directory Server.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**IdM servers** role `idm_server` **IdM replicas** role `idm_configure_replica` **IdM clients** role `idm_client` **IdM DNS** role `idm_dns` **IdM and AD trust** role `idm_ad_trust` **Directory Server installation** role `install_dirsrv` **Directory Server search** role `ocp_search_dirsrv`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Manage Keycloak (Red Hat build of Keycloak)

Description	Install Red Hat build of Keycloak and manage realms, clients, client scopes, groups, users, mappers, identity providers and federation, and connect NetBox to it.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Keycloak installation** role `install_rhbk` **Realms** role `rhbk_realm` **Clients** role `rhbk_client` **Client scopes** role `rhbk_client_scope` **Groups** role `rhbk_groups` **Users** role `rhbk_users` **Mappers** role `rhbk_setup_mapper` **Identity providers** role `rhbk_manage_idp` **Federation** role `rhbk_manage_federation` **NetBox OIDC login** role `netbox_oidc`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Deploy observability, logging and monitoring

Description	Install Grafana and manage its data sources, folders, dashboards and alerts, deploy Elasticsearch, Loki, OpenShift logging and Zabbix, and export login and workspace metrics.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Grafana on RHEL** role `install_grafana` **Grafana on OpenShift** role `grafana_install` **Grafana data sources** role `grafana_create_datasource` **Grafana folders** role `grafana_manage_folders` **Grafana dashboards** role `grafana_upload_dashboards` **Grafana alerts** role `grafana_upload_alerts` **Grafana data sources playbook** playbook `grafana.ado-grafana-datasources` **Grafana folders playbook** playbook `grafana.ado-grafana-folders` **Grafana dashboards playbook** playbook `grafana.ado-grafana-dashboards` **Grafana alerts playbook** playbook `grafana.ado-grafana-alerts` **Grafana content sync playbook** playbook `grafana.ado-grafana-content` **Elasticsearch installation** role `install_elastic` **Elasticsearch and Kibana** role `elastic` **OpenShift logging** role `ocp_logging` **Loki** role `ocp_loki` **Zabbix monitoring stack** role `ocp_zabbix` **Keycloak login events exporter** role `keycloak_login_events_exporter` **Keycloak login events exporter playbook** playbook `rhbk.ado-rhbk-login-events-exporter` **Dev Spaces status exporter playbook** playbook `devspaces.ado-devspaces-status-exporter`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Prepare OpenShift clusters and operators

Description	Subscribe and wait for operators, create namespaces and service accounts, configure ingress certificates, routes, image registry, pull secrets and console settings, and clean up stuck resources.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Operator defaults** role `ocp_operator_defaults` **Operator subscriptions** role `ocp_operator_subscription` **Operator groups** role `ocp_operatorgroups` **Wait for operators** role `ocp_wait_operator` **Wait for pods** role `ocp_wait_pods` **Namespaces** role `ocp_namespace` **Service accounts** role `ocp_service_accounts` **Pull secrets** role `ocp_pull_secrets` **Secret replication to Vault** role `ocp_secret_replicator` **Integrated image registry** role `ocp_image_registry` **Default ingress certificate** role `ocp_default_ingress_cert` **cert-manager** role `ocp_cert_manager` **AWS Private CA issuer** role `ocp_awspca` **Routes** role `ocp_routes` **Alternate routes** role `ocp_alt_routes` **Component routes** role `ocp_component_route` **Route discovery** role `ocp_discover_routes` **Console banner** role `ocp_console_banner` **Web Terminal** role `ocp_web_terminal` **Descheduler** role `ocp_descheduler` **CRD printing** role `ocp_print_crd` **Absent namespace cleanup** role `ocp_absent_cleanup`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Configure OpenShift networking and storage

Description	Manage network policies, tenant and user-defined networks, egress, MetalLB, NMState, NFS, iSCSI and EFS storage, MinIO object storage, backup with OADP and data foundation.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**Networking orchestration** role `ocp_networking` **Network policies** role `ocp_network_policy` **User-defined networks** role `ocp_udn` **Egress IP and firewall** role `ocp_egress` **MetalLB** role `ocp_metallb_install` **NMState** role `ocp_nmstate_install` **NFS storage** role `ocp_nfs_storage` **iSCSI storage** role `ocp_iscsi_storage` **EFS CSI driver** role `ocp_efs_csi` **MinIO** role `ocp_minio` **OADP backup** role `ocp_oadp` **Data foundation** role `ocp_data_foundation`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Secure OpenShift access and compliance

Description	Configure LDAP, OIDC and htpasswd authentication, manage Keycloak client secrets, deploy Advanced Cluster Security with policies and reports, and run compliance scans and remediation.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**LDAP authentication** role `ocp_ldap_auth` **OIDC authentication** role `ocp_oidc_auth` **htpasswd administrators** role `ocp_htpasswd_admin` **Keycloak client secrets** role `ocp_rhbk_client_secrets` **Advanced Cluster Security** role `ocp_acs` **ACS policies** role `acs_upload_policies` **ACS vulnerability reports** role `acs_report` **Compliance Operator installation** role `ocp_compliance_install` **Compliance profiles** role `ocp_compliance_profiles` **Compliance scans** role `ocp_compliance_scan` **Compliance remediation** role `ocp_compliance_remediation`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Deploy developer tools and applications on OpenShift

Description	Deploy GitLab and runners, Developer Hub, Dev Spaces, GitOps applications, Quay, BookStack, Pega, Kafka, Postfix and Jira content.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**GitLab on RHEL** role `install_gitlab` **GitLab on OpenShift** role `gitlab_install` **GitLab Runner** role `ocp_gitlab_runner` **Developer Hub** role `ocp_dev_hub` **Dev Spaces** role `ocp_devspaces` **Dev Spaces user configuration** role `ocp_devspaces_user_config` **GitOps** role `ocp_gitops` **GitOps applications** role `ocp_gitops_application` **Quay** role `ocp_quay` **BookStack** role `bookstack_openshift` **Pega** role `ocp_pega` **Kafka** role `kafka_install` **Postfix** role `install_postfix` **Jira** role `jira` **Jira stories** role `jira_stories`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Use Case: Run virtualization, multi-cluster and disconnected installs

Description	Install OpenShift Virtualization and the Migration Toolkit, build VM images, manage clusters with ACM policies and validate disconnected install bundles.
Collection	`infra.automation_development_office` (Validated, Red Hat)
Key content	**OpenShift Virtualization** role `ocp_virtualization` **OpenShift Virtualization HyperConverged** role `ocp_virtualization_install` **Migration Toolkit for Virtualization** role `ocp_mtv` **VM image creation** role `vm_image_management` **Advanced Cluster Management** role `ocp_acm` **ACM policy delivery manifests** role `ocp_acm_policy` **Disconnected install validation** role `ocp_disconnected_install` **Disconnected bundle validation playbook** playbook `openshift.ado-validate-disconnected-install-bundle`
Notes	• A collection of roles and playbooks that stand up supporting platform services (AAP, Satellite, identity, observability, developer tools) mostly on OpenShift and RHEL.

Ansible Automation Platform (controller configuration as code)

`infra.controller_configuration`

Validated Red Hat · version 3.4.3 · updated 2026-10-01 · 1 modules → 7 use cases

Use Case: Set up organizations, teams, users and permissions

Description	Create organizations, teams, users and role assignments, OAuth applications and labels on the automation controller.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Organizations role** role `organizations` **Teams role** role `teams` **Users role** role `users` **Role assignments role** role `roles` **OAuth applications role** role `applications` **Labels role** role `labels`

Use Case: Manage credentials and execution environments

Description	Create credential types, credentials and external credential input sources, and register execution environments.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Credential types role** role `credential_types` **Credentials role** role `credentials` **Credential input sources role** role `credential_input_sources` **Execution environments role** role `execution_environments`

Use Case: Manage inventories, hosts and instances

Description	Create inventories, groups, hosts and inventory sources, update sources, bulk-create hosts, and register instances and instance groups.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Inventories role** role `inventories` **Groups role** role `groups` **Hosts role** role `hosts` **Bulk host creation role** role `bulk_host_create` **Inventory sources role** role `inventory_sources` **Inventory source update role** role `inventory_source_update` **Instances role** role `instances` **Instance groups role** role `instance_groups`

Use Case: Define projects, job templates and workflows

Description	Create projects, job templates, workflow job templates, schedules and notification templates, and refresh projects.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Projects role** role `projects` **Project update role** role `project_update` **Job templates role** role `job_templates` **Workflow job templates role** role `workflow_job_templates` **Schedules role** role `schedules` **Notification templates role** role `notification_templates`

Use Case: Launch and control automation

Description	Launch jobs, workflows and bulk jobs, run or cancel ad hoc commands and jobs, and collect asynchronous task status.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Job launch role** role `job_launch` **Workflow launch role** role `workflow_launch` **Bulk job launch role** role `bulk_job_launch` **Job cancellation role** role `jobs_cancel` **Ad hoc command role** role `ad_hoc_command` **Ad hoc command cancellation role** role `ad_hoc_command_cancel` **Async status role** role `collect_async_status`

Use Case: Configure the controller platform

Description	Configure settings and licences after installation, dispatch object creation across roles, share global variables, check dependencies, and gather statistics.

Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**Settings role** role `settings` **Licence role** role `license` **Dispatch role** role `dispatch` **Global variables role** role `global_vars` **Dependency check role** role `meta_dependency_check` **Post-installation configuration** playbook `configure_controller` **Controller statistics** playbook `get_stats`

Use Case: Export, compare and refactor configuration as code

Description	Export controller objects to a file tree and recreate them from it, compare controller state with code, and bulk-rename objects or set their organization.
Collection	`infra.controller_configuration` (Validated, Red Hat)
Key content	**File tree export role** role `filetree_create` **File tree import role** role `filetree_read` **Object diff role** role `object_diff` **Export diff** `controller_export_diff` **Object diff lookup** lookup `controller_object_diff` **Flatten file tree output** playbook `flatten_filetree_create_output` **Rename objects** playbook `rename_objects` **Set organization** playbook `set_organization`

Event-Driven Ansible Controller `infra.eda_configuration`

Validated Red Hat · version 1.1.0 · updated 2024-05-07 · 7 modules → 3 use cases

Use Case: Configure rulebook activations and their content

Description	Manage credentials, decision environments, projects with syncs, and rulebook activations in EDA Controller, as modules or as roles.
Collection	`infra.eda_configuration` (Validated, Red Hat)
Key content	**Credentials** `credential` **Decision environments** `decision_environment` **Projects** `project` **Project syncs** `project_sync` **Rulebook activations** `rulebook_activation` **Credentials role** role `credential` **Decision environments role** role `decision_environment` **Projects role** role `project` **Project syncs role** role `project_sync` **Rulebook activations role** role `rulebook_activation`

Use Case: Manage users and tokens

Description	Manage EDA Controller users and user tokens.
Collection	`infra.eda_configuration` (Validated, Red Hat)
Key content	**Users** `user` **User tokens** `user_token` **Users role** role `user` **User tokens role** role `user_token`

Use Case: Apply and query configuration as code

Description	Run all object roles for which variables are defined and search the EDA API.
Collection	`infra.eda_configuration` (Validated, Red Hat)
Key content	**Configuration dispatch** role `dispatch` **EDA API lookup** lookup `eda_api`

Ansible execution environments `infra.ee_utilities`

Validated Red Hat · version 4.6.0 · updated 2026-09-17 · 0 modules → 1 use cases

Use Case: Build execution environments and migrate virtualenvs

Description	Build custom execution environments and compile Python requirements from legacy AAP 1.2 virtualenvs.
Collection	`infra.ee_utilities` (Validated, Red Hat)
Key content	**Execution environment builder** role `ee_builder` **Create execution environment playbook** playbook `create_execution_environment` **Virtualenv migration** role `virtualenv_migrate`

Red Hat Support `infra.support_assist`

Validated Red Hat · version 2.1.0 · updated 2026-10-18 · 0 modules → 2 use cases

Use Case: Open and update Red Hat support cases

Description	Create cases, add comments and attach files through the Red Hat API, and refresh API tokens.
Collection	`infra.support_assist` (Validated, Red Hat)

Key content	**Case management** role `rh_case` **Red Hat token refresh** role `rh_token_refresh` **Create case** playbook `rh_case_create` **Update case** playbook `rh_case_update`

Use Case: Collect diagnostics from RHEL, OpenShift and AAP

Description	Gather SOS reports from hosts and OpenShift nodes, OpenShift must-gather data and AAP API data.
Collection	`infra.support_assist` (Validated, Red Hat)
Key content	**SOS reports** role `sos_report` **OpenShift must-gather** role `ocp_must_gather` **AAP API gather** role `aap_api_gather` **AAP API token** role `aap_api_token` **SOS report** playbook `sos_report` **RHEL host SOS report** playbook `sos_report_rhel` **OpenShift node SOS report** playbook `sos_report_ocp` **OpenShift must-gather pipeline** playbook `ocp_must_gather` **AAP API gather and case upload** playbook `aap_api_gather`

Other certified and validated collections

Module-level content for these collections could not be read from a public source repository. Browse them on Automation Hub.

`ansible.tower`	Ansible content that interacts with the AWX or Ansible Tower API. (Certified, Red Hat; Legacy collection for Ansible Tower 3.8 (last updated 2022). Use ansible.controller for Automation Platform 2.x.)